

Combinatorics of minimal absent words for a sliding window

Tooru Akagi¹, Yuki Kuhara¹, Takuya Mieno^{1,2}, Yuto Nakashima¹,
Shunsuke Inenaga^{1,3}, Hideo Bannai⁴, and Masayuki Takeda¹

¹*Department of Informatics, Kyushu University, Japan*
{toru.akagi@inf.kyushu-u.ac.jp, takuya.mieno@inf.kyushu-u.ac.jp,
yuto.nakashima, inenaga, takeda}@inf.kyushu-u.ac.jp

²*Japan Society for the Promotion of Science, Japan*

³*PRESTO, Japan Science and Technology Agency, Japan*

⁴*M&D Data Science Center, Tokyo Medical and Dental University, Japan*
hdbn.dsc@tmd.ac.jp

Abstract

A string w is called a minimal absent word (MAW) for another string T if w does not occur in T but the proper substrings of w occur in T . For example, let $\Sigma = \{a, b, c\}$ be the alphabet. Then, the set of MAWs for string $w = abaab$ is $\{aaa, aaba, bab, bb, c\}$. In this paper, we study combinatorial properties of MAWs in the sliding window model, namely, how the set of MAWs changes when a sliding window of fixed length d is shifted over the input string T of length n , where $1 \leq d < n$. We present *tight* upper and lower bounds on the maximum number of changes in the set of MAWs for a sliding window over T , both in the cases of general alphabets and binary alphabets. Our bounds improve on the previously known best bounds [Crochemore et al., 2020].

1 Introduction

We say that a string s *occurs* in another string T if s is a substring of T . A non-empty string w is said to be a *minimal absent word* (an *MAW*) for a string T if w does not occur in T but any proper substring of w occurs in T . Note that by definition a string of length 1 (namely a character) which does not occur in T is also an MAW for T . On the other hand, any MAW for T of length at least 2 can be represented as aub , where a and b are single characters and u is a (possibly empty) string, such that both au and ub occur in T . For example, let $\Sigma = \{a, b, c\}$ be the alphabet. Then, the set of MAWs for string $w = abaab$ is $\{aaa, aaba, bab, bb, c\}$.

Applications of (minimal) absent words include phylogeny [6], data compression [12, 13], musical information retrieval [9], and bioinformatics [1, 7, 19, 16].

1.1 Algorithms for finding MAWs for string

Given the afore-mentioned motivations, finding MAWs from a given string has been an important and interesting string algorithmic problem and several nice solutions have been

proposed. The first non-trivial algorithm, which was given by Crochemore et al. [11], finds the set $\text{MAW}(T)$ of all MAWs for a given string T of length n over an alphabet of size σ in $\Theta(\sigma n)$ time with $O(n)$ working space. Since $|\text{MAW}(T)| = O(\sigma n)$ for any string T of length n and $|\text{MAW}(S)| = \Omega(\sigma n)$ for some string S of length n [11], Crochemore et al.'s algorithm [11] runs in optimal time in the worst case. Fujishige et al. [15] improved Crochemore et al.'s algorithm so that $\text{MAW}(T)$ can be computed in output-sensitive $O(n + |\text{MAW}(T)|)$ time with $O(n)$ working space. Both of these algorithms use the *directed acyclic word graph* (DAWG) [5] of string T as a powerful tool for enumerating all MAWs for T . Charalampopoulos et al. [8] presented an algorithm that computes all MAWs in output-sensitive time without using the DAWG of T . Belazzougui et al. [4] showed that $\text{MAW}(T)$ can also be computed in $O(n + |\text{MAW}(T)|)$ time, provided that the *bidirectional Burrows-Wheeler transform* of a given string has already been computed. Barton et al. [2] proposed a practical algorithm to compute $\text{MAW}(T)$ in $\Theta(n\sigma)$ time and working space¹ based on the *suffix array* [17] of T . A parallel algorithm for computing MAWs has also been proposed [3]. Fici and Gawrychowski [14] extended the notion of MAWs to rooted/unrooted labeled trees and presented efficient algorithms to compute them.

1.2 MAWs for sliding window

This paper follows the recent line of research on *MAWs for the sliding window model*, which was initiated by Crochemore et al. [10]. In this model, the goal is to compute or analyze $\text{MAW}(T[i..i + d - 1])$ for every window $T[i..i + d - 1]$ of fixed length $d \geq 1$ that shifts T from left to right with increasing $i = 1, \dots, n - d + 1$.

Crochemore et al. [10] presented a suffix-tree based algorithm that maintains the set of all MAWs for a sliding window in $O(\sigma n)$ time using $O(\sigma d)$ working space. Crochemore et al. [10] also showed how their algorithm can be applied to approximate pattern matching under the *length weighted index* (LWI) metric [6].

The (in)efficiency of their algorithms is heavily dependent on combinatorial properties of MAWs for the sliding window. In particular, Crochemore et al. [10] studied the number of MAWs to be added/deleted when the current window is shifted to the right by one character. As was done in [10], for ease of discussion let us separately consider

- adding a new character $T[i + d]$ to the current window $T[i..i + d - 1]$ of length d which forms $T[i..i + d]$, and
- deleting the left-most character $T[i - 1]$ from the current window $T[i - 1..i + d - 1]$ which forms $T[i..i + d - 1]$ of length d .

We remark that these two operations are symmetric.

Crochemore et al. [10] considered how many MAWs can change before and after the window has been shifted by one position, and showed that

$$\begin{aligned} |\text{MAW}(T[i..i + d]) \triangle \text{MAW}(T[i..i + d - 1])| &\leq (s_i - s_\alpha)(\sigma - 1) + \sigma + 1, \\ |\text{MAW}(T[i - 1..i + d - 1]) \triangle \text{MAW}(T[i..i + d - 1])| &\leq (p_i - p_\beta)(\sigma - 1) + \sigma + 1, \end{aligned}$$

where $\triangle$ denotes the symmetric difference and

¹The original claimed bound in [2] is $O(n)$, however, the authors assumed that $\sigma = O(1)$.

- s_i is the length of the longest repeating suffix of $T[i..i + d - 1]$,
- s_α is that of the longest suffix of $T[i..i + d - 1]$ having an internal occurrence immediately followed by $\alpha = T[i + d]$,
- p_i is that of the longest repeating prefix of $T[i..i + d - 1]$, and
- p_β is that of the longest prefix of $T[i..i + d - 1]$ having an internal occurrence immediately preceded by $\beta = T[i - 1]$.

Since both $s_i - s_\alpha$ and $p_i - p_\beta$ can be at most $d - 1$ in the worst case, the asymptotic bounds for the numbers of changes in the set of MAWs obtained by Crochemore et al. [10] are:

$$\begin{aligned} |\text{MAW}(T[i..i + d]) \triangle \text{MAW}(T[i..i + d - 1])| &\in O(\sigma d), \\ |\text{MAW}(T[i - 1..i + d - 1]) \triangle \text{MAW}(T[i..i + d - 1])| &\in O(\sigma d). \end{aligned} \quad (1)$$

Crochemore et al. [10] also considered the *total changes* in the set of MAWs for every sliding window over the string T , and showed that

$$\sum_{i=1}^{n-d} (|\text{MAW}(T[i..i + d - 1]) \triangle \text{MAW}(T[i + 1..i + d])|) \in O(\sigma n). \quad (2)$$

1.3 Our contribution

The goal of this paper is to give more rigorous analyses on the number of MAWs for the sliding window model. This study is well motivated since revealing more combinatorial insights to the sets of MAWs for the sliding windows can lead to more efficient algorithms for computing them.

In this paper, we first give the following upper bounds:

$$\begin{aligned} |\text{MAW}(T[i..i + d]) \triangle \text{MAW}(T[i..i + d - 1])| &\leq d + \sigma' + 1, \\ |\text{MAW}(T[i - 1..i + d - 1]) \triangle \text{MAW}(T[i..i + d - 1])| &\leq d + \sigma' + 1, \end{aligned} \quad (3)$$

where σ' is the number of distinct characters in $T[i..i + d - 1]$. We then show that our new upper bounds in (3) are *tight* by showing a family of strings achieving these bounds.

Since $\sigma' \leq d$ always holds, we immediately obtain new asymptotic upper bounds

$$\begin{aligned} |\text{MAW}(T[i..i + d]) \triangle \text{MAW}(T[i..i + d - 1])| &\in O(d), \\ |\text{MAW}(T[i - 1..i + d - 1]) \triangle \text{MAW}(T[i..i + d - 1])| &\in O(d). \end{aligned} \quad (4)$$

Our new upper bounds in (4) improve Crochemore et al.'s upper bounds in (1) for any alphabet of size $\sigma \in \omega(1)$. Our upper bounds in (4) are also *tight* as there exists a family of strings achieving the matching lower bounds $\Omega(d)$.

In this paper, we also present a new upper bound for the total changes of MAWs:

$$\sum_{i=1}^{n-d} (|\text{MAW}(T[i..i + d - 1]) \triangle \text{MAW}(T[i + 1..i + d])|) \in O(\min\{\sigma, d\}n) \quad (5)$$

which improves the previous bound $O(\sigma n)$ in (2). We then show that this new upper bound in (5) is also *tight*.

All of our new bounds afore-mentioned are tight for any alphabet of size $\sigma' \geq 3$. We further explore the case of binary alphabets with $\sigma' = 2$, and show that there exist even tighter bounds in the binary case. Namely, for $\sigma' = 2$, we prove that

$$\begin{aligned} |\text{MAW}(T[i..i+d]) \triangle \text{MAW}(T[i..i+d-1])| &\leq \max\{3, d\}, \\ |\text{MAW}(T[i-1..i+d-1]) \triangle \text{MAW}(T[i..i+d-1])| &\leq \max\{3, d\}. \end{aligned} \quad (6)$$

We remark that plugging $\sigma' = 2$ into (3) for the general case only gives $d + \sigma' + 1 = d + 3$, which is larger than $\max\{3, d\}$ in (6). We consider the case $\sigma' \geq d$ in Lemmas 10 and 11. We also show that the upper bounds $\max\{3, d\}$ in (6) are *tight* by giving the matching lower bounds with a family of binary strings.

A part of the results reported in this article appeared in a preliminary version of this paper [18]. In addition, this present article considers the case of binary alphabets and presents tight upper and lower bounds for this case.

2 Preliminaries

2.1 Strings

Let Σ be an alphabet. An element of Σ is called a character. An element of Σ^* is called a string. The length of a string T is denoted by $|T|$. The empty string ε is the string of length 0. If $T = xyz$, then x , y , and z are called a *prefix*, *substring*, and *suffix* of T , respectively. They are called a *proper prefix*, *proper substring*, and *proper suffix* of T if $x \neq T$, $y \neq T$, and $z \neq T$, respectively.

For any $1 \leq i \leq |T|$, the i -th character of T is denoted by $T[i]$. For any $1 \leq i \leq j \leq |T|$, $T[i..j]$ denotes the substring of T starting at i and ending at j . For convenience, let $T[i..j] = \varepsilon$ for $0 \leq j < i \leq |T| + 1$. For any $i \leq |T|$ and $1 \leq j$, let $T[..i] = T[1..i]$ and $T[j..] = T[j..|T|]$.

We say that a string w *occurs* in a string T if w is a substring of T . Note that by definition the empty string ε is a substring of any string T and hence ε always occurs in T .

2.2 Minimal absent words (MAWs)

A string w is called an *absent word* for a string T if w does not occur in S . An absent word w for S is called a *minimal absent word* or *MAW* for S if any proper substring of w occurs in S . We denote by $\text{MAW}(S)$ the set of all MAWs for S . By the definition of MAWs, it is clear that $w \in \text{MAW}(S)$ iff the three following conditions hold:

- (A) w does not occur in S ;
- (B) $w[2..]$ occurs in S ;
- (C) $w[..|w| - 1]$ occurs in S .

We note that if w is a string of length 1 which does not occur in S (i.e. w is a single character in Σ of size σ not occurring in S), then w is a MAW for T since $w[2..] = w[..|w| - 1] = \varepsilon$ is a substring of S .

Example 1. Let $\Sigma = \{a, b, c, d\}$. Then, the MAW of the string $T = \text{cbaaaa}$ is :

$$\text{MAW}(\text{cbaaaa}) = \{cc, bb, aaaaa, bc, ab, ca, ac, d\},$$

2.3 MAWs for a sliding window

Given a string T of length n and a sliding window $S_i = T[i..j]$ of length $d = j - i + 1$ for increasing $i = 1, \dots, n - d + 1$, our goal is to analyze how many MAWs for the sliding window can change when the window shifts over the string T . We will consider both the maximum change per one shift, and the maximum total number of changes when sliding the window from the beginning to the end.

As was done in [10], for simplicity, we separately consider two symmetric operations of appending a new character to the right of the window and of deleting the leftmost character from the window.

Example 2. Let $\Sigma = \{a, b, c, d\}$. Consider to append character c to the right of string $cbaaaa$. Then,

$$\begin{aligned} \text{MAW}(cbaaaa) &= \{cc, bb, aaaaa, bc, ab, ca, ac, d\}, \\ \text{MAW}(cbaaaac) &= \{cc, bb, aaaaa, bc, ab, ca, acb, bac, baac, baaac, d\}. \end{aligned}$$

Thus $\text{MAW}(cbaaaa) \triangle \text{MAW}(cbaaaac) = \{\underline{ac}, acb, bac, baac, baaac\}$, where the underlined string is deleted from and the strings without underlines are added to the set of MAWs by appending c to $cbaaaa$.

3 Tight bounds on the changes to MAWs for sliding window

In this section, we present our new bounds for the changes of MAWs for the sliding window over the string T . In Section 3.1, we consider the number of changes of MAWs when the current window $T[i..j]$ is extended by adding a new character $T[j+1]$. Section 3.2 is for the symmetric case where the leftmost character $T[i]$ is deleted from $T[i..j+1]$. Finally, in Section 3.3, we consider the total number of changes of MAWs while the window has been shifted from the beginning of T until its end.

3.1 Changes to MAWs when a character is appended to the right

We consider the number of changes of MAWs when appending $T[j+1]$ to the current window $T[i..j]$.

For the number of deleted MAWs, the next lemma is known:

Lemma 1 ([10]). *For any $1 \leq i \leq j < n$, $|\text{MAW}(T[i..j]) \setminus \text{MAW}(T[i..j+1])| = 1$.*

Next, we consider the number of added MAWs. We classify each MAW w in $\text{MAW}(T[i..j+1]) \setminus \text{MAW}(T[i..j])$ to the following three types² (see Figure 1). A MAW w in $\text{MAW}(T[i..j+1]) \setminus \text{MAW}(T[i..j])$ is said to be of:

Type 1 if neither $w[2..]$ nor $w[..|w|-1]$ occurs in $T[i..j]$;

Type 2 if $w[2..]$ occurs in $T[i..j]$ but $w[..|w|-1]$ does not occur in $T[i..j]$;

Type 3 if $w[2..]$ does not occur in $T[i..j]$ but $w[..|w|-1]$ occurs in $T[i..j]$.

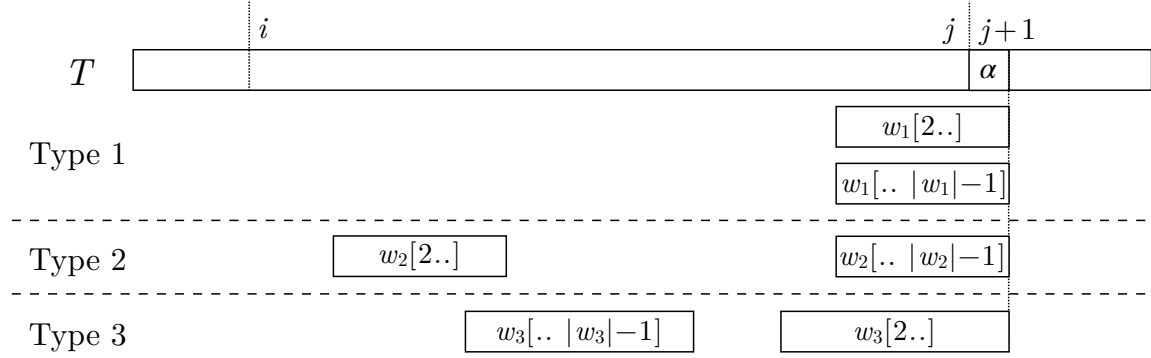


Figure 1: Illustration for the three types of MAWs, where $w_1 \in \mathcal{M}_1$, $w_2 \in \mathcal{M}_2$, and $w_3 \in \mathcal{M}_3$.

We denote by $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$ the sets of MAWs of Type 1, Type 2 and Type 3, respectively. Recall that w is a MAW for $T[i..j+1]$.

Let $\sigma_{i,j}$ be the number of distinct characters occurring in the current window $T[i..j]$. We also use $\sigma' = \sigma_{i,j}$ for simplicity.

The next three lemmas show the upper bounds of $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$:

Lemma 2 ([10]). *For any $1 \leq i \leq j < n$, $|\mathcal{M}_1| \leq 1$. Also, if α is the character appended to $T[i..j]$, then the only element of $\mathcal{M}_1$ is of form α^k with some $k \geq 1$.*

Lemma 3. *For any $1 \leq i \leq j < n$, $|\mathcal{M}_2| \leq \sigma'$.*

Proof. It is shown in [10] that the last characters of all MAWs in $\mathcal{M}_2$ are all distinct. Furthermore, by the definition of $\mathcal{M}_2$, the last character $T[j+1]$ of each MAW in $\mathcal{M}_2$ must occur in the current window $T[i..j]$. Thus, $|\mathcal{M}_2| \leq \sigma'$. $\square$

Lemma 4. *For any $1 \leq i \leq j < n$, $|\mathcal{M}_3| \leq d - 1$, where $d = j - i + 1$.*

Proof. We show that there is an injection $f : \mathcal{M}_3 \rightarrow [i, j - 1]$ which maps each MAW $w \in \mathcal{M}_3$ to the ending position of the leftmost occurrence of $w[..|w|-1]$ in the current window $T[i..j]$.

First, we show that the range of this function f is $[i, j - 1]$. By definition, w is absent from $T[i..j+1]$ and $w[|w|] = T[j+1]$ for each $w \in \mathcal{M}_3$, and thus, no occurrence of $w[..|w|-1]$ in $T[i..j]$ ends at position j . Hence, the range of f does not contain the position j , i.e. it is $[i, j - 1]$.

Next, for the sake of contradiction, we assume that f is not an injection, i.e. there are two distinct MAWs $w_1, w_2 \in \mathcal{M}_3$ such that $f(w_1) = f(w_2)$. Without loss of generality, assume $|w_1| \geq |w_2|$. Since $w_1[|w_1|] = w_2[|w_2|] = T[j+1]$ and $f(w_1) = f(w_2)$, w_2 is a suffix of w_1 . If $|w_1| = |w_2|$, then $w_1 = w_2$ and it contradicts with $w_1 \neq w_2$. If $|w_1| > |w_2|$, then w_2 is a proper suffix of w_1 , and it contradicts with the fact that w_2 is absent from $T[i..j+1]$ (see Figure 2). Therefore, f is an injection and $|\mathcal{M}_3| \leq j - 1 - i + 1 = d - 1$. $\square$

Summing up all the upper bounds for $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$, we obtain the following:

²At least one of $w[2..]$ and $w[..|w|-1]$ does not occur in $T[i..j]$, since $w \notin \text{MAW}(T[i..j])$.

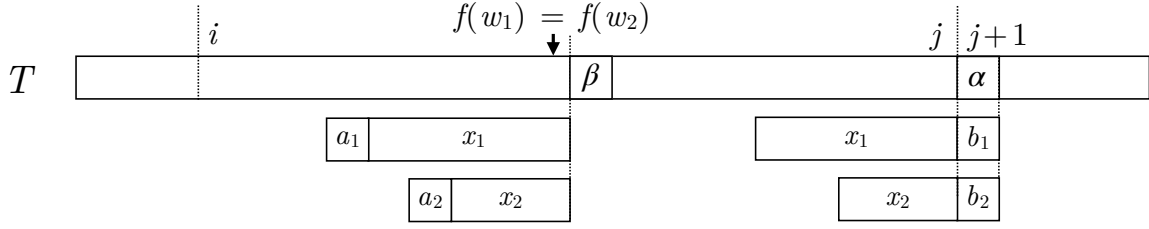


Figure 2: Illustration for the contradiction in the proof of Lemma 5. Consider two strings $w_1 = a_1x_1b_1$ and $w_2 = a_2x_2b_2$ that are MAWs for T of Type 3 where $a_1, a_2, b_1, b_2 \in \Sigma$ and $x_1, x_2 \in \Sigma^*$. If $|w_1| > |w_2|$ and $f(w_1) = f(w_2)$, then x_2 is a proper suffix of x_1 , and it contradicts that $a_2x_2b_2$ is absent from T .

Lemma 5. For any $1 \leq i \leq j < n$, $|\text{MAW}(T[i..j+1]) \setminus \text{MAW}(T[i..j])| \leq \sigma' + d$, where $d = j - i + 1$.

Proof. Immediately follows from Lemmas 2, 3, and 4 and that $\mathcal{M}_1$, $\mathcal{M}_2$, and $\mathcal{M}_3$ are mutually disjoint. $\square$

Now we obtain the main result of this subsection, which shows the matching upper and lower bounds for $|\text{MAW}(T[i..j+1]) \triangle \text{MAW}(T[i..j])|$.

Theorem 1. For any $1 \leq i \leq j < n$, $|\text{MAW}(T[i..j+1]) \triangle \text{MAW}(T[i..j])| \leq \sigma' + d + 1$, where $d = j - i + 1$. The upper bound is tight when $\sigma \geq 3$ and $\sigma' + 1 \leq \sigma$.

Proof. By Lemma 1 and Lemma 5, we have $|\text{MAW}(T[i..j+1]) \triangle \text{MAW}(T[i..j])| = |\text{MAW}(T[i..j+1]) \setminus \text{MAW}(T[i..j])| + |\text{MAW}(T[i..j]) \setminus \text{MAW}(T[i..j+1])| \leq \sigma' + d + 1$.

In the following, we show that the upper bound is tight, i.e. there is a string Z of length d and a character α where $|\text{MAW}(Z) \triangle \text{MAW}(Z\alpha)| = \sigma' + d + 1$ for any two integers d and σ' with $1 \leq \sigma' \leq d$ and $\sigma' + 1 \leq \sigma$. Let $\Sigma = \{a_1, a_2, \dots, a_\sigma\}$ be an alphabet. Given two integers d and σ' with $1 \leq \sigma' \leq d$ and $\sigma' + 1 \leq \sigma$, consider a string $Z = a_1a_2 \dots a_{\sigma'-1}a_{\sigma'}^{d-\sigma'+1}$ of length d and a character $\alpha = a_{\sigma'+1}$. Then,

$$\text{MAW}(Z) \setminus \text{MAW}(Z\alpha) = \{\alpha\}.$$

Also,

$$\begin{aligned} \text{MAW}(Z\alpha) \setminus \text{MAW}(Z) &= \{\alpha^2\} \cup \{\alpha a_i \mid 1 \leq i \leq \sigma'\} \cup \{a_i \alpha \mid 1 \leq i \leq \sigma' - 1\} \\ &\quad \cup \{a_{\sigma'-1}a_{\sigma'}^e \alpha \mid 1 \leq e \leq d - \sigma'\}. \end{aligned}$$

This leads to the matching lower bound $|\text{MAW}(Z) \triangle \text{MAW}(Z\alpha)| = \sigma' + d + 1$. $\square$

A concrete example for our lower-bound strings Z and $Z\alpha$ is shown below.

Example 3. Let $Z = \text{abcddd}$ where $d = |Z| = 6$, $\sigma' = 4$, and $d - \sigma' + 1 = 3$. Also, let $\alpha = \text{e}$. Then,

$$\text{MAW}(\text{abcddd}) \setminus \text{MAW}(\text{abcddde}) = \{\text{e}\}$$

and

$$\begin{aligned}
& \text{MAW}(\text{abcddde}) \setminus \text{MAW}(\text{abcddd}) \\
&= \mathcal{M}_1 \cup \mathcal{M}_2 \cup \mathcal{M}_3 \\
&= \{\text{ee}\} \cup \{\text{ea}, \text{eb}, \text{ec}, \text{ed}\} \cup \{\text{ae}, \text{be}, \text{ce}, \text{cde}, \text{cdde}\},
\end{aligned}$$

and therefore $|\text{MAW}(Z) \triangle \text{MAW}(Z\alpha)| = \sigma' + d + 1 = 11$.

3.1.1 Changes to MAWs when a character already occurring in the window is added to the right

In this subsection, we consider the case where a new character $T[j+1]$ that is appended to the right of the current window $T[i..j]$ already occurs in $T[i..j]$. This means that $\sigma_{i,j} = \sigma_{i,j+1}$, i.e., the alphabet size does not increase before and after the new character is added.

The next lemma shows that a conflict occurs between $\mathcal{M}_1$ and $\mathcal{M}_2$ when $\sigma_{i,j} = \sigma_{i,j+1}$.

Lemma 6. *For any $T[i..j]$ such that $d = j - i + 1 \geq 3$ and $\sigma_{i,j} = \sigma_{i,j+1}$, $|\mathcal{M}_1| + |\mathcal{M}_2| \leq \sigma_{i,j}$.*

Proof. Let $\mathbf{c} = T[j+1]$ and let k be the length of the maximum run of $\mathbf{c}$'s that is the suffix of $T[i..j]$. If $T[j] \neq \mathbf{c}$ then let $k = 0$. By the definition of $\mathcal{M}_1$, $\mathbf{c}^{k+2}$ is the only candidate for the Type-1 MAW for $T[i..j+1]$, in which case $au = ub = \mathbf{c}^{k+1}$ occurs only once in $T[i..j+1]$ as a suffix. This means that $\mathbf{c}^{k+2}$ can be a Type-1 MAW for $T[i..j+1]$ only if $\mathbf{c}^k$ is the longest run of $\mathbf{c}$'s in $T[i..j]$.

Now suppose that $\mathbf{c}^{k+2}$ is a Type-1 MAW for $T[i..j+1]$, and let $a'u'\mathbf{c}$ denote a Type-2 MAW for $T[i..j+1]$. Then, by definition, $a'u'$ occurs only once in $T[i..j+1]$ as a suffix (see also the middle of Figure 1).

- If $|u'| \geq k$, then $\mathbf{c}^{k+1}$ is a suffix of u' as shown in Figure 5. However, by the definition of Type-2 MAWs, $u'\mathbf{c}$ must occur in S (see also the middle of Figure 1), which implies that $\mathbf{c}^{k+1}$ occurs in S . This contradicts that $\mathbf{c}^k$ is the longest run of $\mathbf{c}$'s in S .
- If $|u'| < k$, then $a'u'\mathbf{c} = \mathbf{c}^{|a'u'\mathbf{c}|}$ with $|a'u'\mathbf{c}| \leq k+1$ occurs in $T[i..j+1]$ as a suffix, and this contradicts that $a'u'\mathbf{c}$ is a MAW for $T[i..j+1]$.

Hence $a'u'\mathbf{c}$ cannot be in $\mathcal{M}_2$, which leads to $|\mathcal{M}_2| \leq \sigma_{i,j} - 1$, and thus $|\mathcal{M}_1| + |\mathcal{M}_2| \leq \sigma_{i,j}$ for any string $T[i..j+1]$ such that $T[i..j]$ contains at least one character that is equal to $T[j+1]$. $\square$

Recall that Lemma 2 and Lemma 3 in the case where $\sigma_{i,j+1} \geq \sigma_{i,j}$ gives us $|\mathcal{M}_1| + |\mathcal{M}_2| \leq \sigma' + 1 = \sigma_{i,j} + 1$. Compared to this, Lemma 6 shaves the total size of $\mathcal{M}_1$ and $\mathcal{M}_2$ by one in the case where $T[j+1]$ already occurs in $T[i..j]$. Coupled with Lemma 4, Lemma 6 leads us to the following corollary:

Corollary 1. *For any $1 \leq i \leq j < n$, $|\text{MAW}(T[i..j+1]) \triangle \text{MAW}(T[i..j])| \leq \sigma' + d$, where $d = j - i + 1$. The upper bound is tight when $\sigma \geq 3$ and $\sigma' = \sigma$.*

3.2 Changes to MAWs when the leftmost character is deleted

Next, we analyze the number of changes of MAWs when deleting the leftmost character from a string. By a symmetric argument to Theorem 1, we obtain:

Corollary 2. *For any $1 < i \leq j \leq n$, $|\text{MAW}(T[i..j]) \triangle \text{MAW}(T[i-1..j])| \leq \sigma' + d + 1$ where $d = j - i + 1$ and σ' is the number of distinct characters occurs in $T[i..j]$. Also, the upper bound is tight when $\sigma \geq 3$ and $\sigma' + 1 \leq \sigma$.*

Proof. Symmetric to the proof of Lemma 1. $\square$

Finally, by combining Theorem 1 and Corollary 2, we obtain the next theorem:

Theorem 2. *Let d be the window length. For any string T of length $n > d$ and each position i in T with $1 \leq i \leq n-d$, $|\text{MAW}(T[i..i+d-1]) \triangle \text{MAW}(T[i+1..i+d])| \in O(d)$. Also, there exists a string T' with $|T'| \geq d+1$ which satisfies $|\text{MAW}(T'[j..j+d-1]) \triangle \text{MAW}(T'[j+1..j+d])| \in \Omega(d)$ for some j with $1 \leq j \leq |T'| - d$.*

This theorem improves Crochemore et al.'s upper bound for $|\text{MAW}(T[i..i+d-1]) \triangle \text{MAW}(T[i+1..i+d])| \in O(\sigma d)$ for any alphabet of size $\sigma \in \omega(1)$.

3.3 Total changes of MAWs when sliding window on string

In this subsection, we consider the total number of changes of MAWs when sliding the window of length d from the beginning of T to the end of T . We denote the total number of changes of MAWs by $\mathcal{S}(T, d) = \sum_{i=1}^{n-d} |\text{MAW}(T[i..i+d-1]) \triangle \text{MAW}(T[i+1..i+d])|$. The following lemma is known:

Lemma 7 ([10]). *For a string T of length $n > d$ over an alphabet Σ of size σ , $\mathcal{S}(T, d) \in O(\sigma n)$.*

The aim of this subsection is to give a more rigorous bound for $\mathcal{S}(T, d)$. We first show that the above bound is tight under some conditions.

Lemma 8. *The upper bound of Lemma 7 is tight when $\sigma \leq d$ and $n - d \in \Omega(n)$.*

Proof. If $\sigma = 2$, the lower bound $\mathcal{S}(T, d) \in \Omega(n - d) = \Omega(\sigma(n - d))$ is obtained by string $T = (\mathbf{ab})^{n/2}$ since $\text{MAW}((\mathbf{ab})^{d/2}) \triangle \text{MAW}((\mathbf{ba})^{d/2}) = \{(\mathbf{ab})^{d/2}, (\mathbf{ba})^{d/2}\}$. In the sequel, we consider the case where $\sigma \geq 3$. Let k be the integer with $(k-1)(\sigma-1) \leq d < k(\sigma-1)$. Note that $k \geq 2$ since $\sigma \leq d$. Let $\Sigma = \{a_1, a_2, \dots, a_\sigma\}$ and $\alpha = a_\sigma$. We consider a string $T' = U^e + U[..m]$ where $U = a_1\alpha^{k-1}a_2\alpha^{k-1}\dots a_{\sigma-1}\alpha^{k-1}$, $e = \lfloor \frac{n}{k(\sigma-1)} \rfloor$, and $m = n \bmod k(\sigma-1)$. Let c be a character that is not equal to α . For any two distinct occurrences $i_1, i_2 \in \text{occ}_{T'}(c)$ for c , $|i_1 - i_2| \geq k(\sigma-1) > d$. Thus, any character $c \neq \alpha$ is absent from at least one of two adjacent windows $T'[i..i+d-1]$ and $T'[i+1..i+d]$ for every $1 \leq i \leq n-d$.

Now we consider a window $W = T'[p-d..p-1]$ where $d+1 \leq p \leq n$ and $T'[p] = \beta \neq \alpha$. Let $\Pi = \{b_1, b_2, \dots, b_{\pi-1}, \alpha\} \subset \Sigma \setminus \{\beta\}$ be the set of all π characters that occur in W . Without loss of generality, we assume that the current window is $W = \alpha^r b_1 \alpha^{k-1} b_2 \alpha^{k-1} \dots b_{\pi-1} \alpha^{k-1}$ and the next window is $W' = W[2..]\beta$ where $r = d \bmod k$ (see Figure 3). For any character $b \in \Pi \setminus \{b_1, b_{\pi-1}, \alpha\}$, $b\alpha^\ell\beta$ is in $\text{MAW}(W') \triangle \text{MAW}(W)$ for

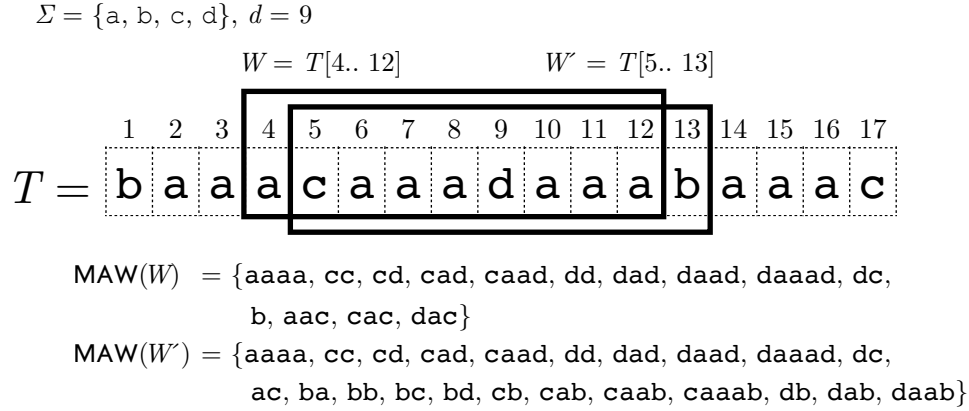


Figure 3: Illustration of examples of MAWs for adjacent two windows. In this example, $\sigma = 4, d = 9$, and $k = 4$. The size of the symmetric difference of $\text{MAW}(W)$ and $\text{MAW}(W')$ is $|\text{MAW}(W) \triangle \text{MAW}(W')| = |\{b, aac, cac, dac, ac, ba, bb, bc, bd, cb, cab, caab, caaab, db, dab, daab\}| = 16$.

every $0 \leq \ell \leq k-1$. If $r > 0$, $b_1 \alpha^\ell \beta$ is also in $\text{MAW}(W') \triangle \text{MAW}(W)$ for every $0 \leq \ell \leq k-1$. Otherwise, b_1 is in $\text{MAW}(W') \triangle \text{MAW}(W)$ and $b_1 \alpha^\ell b_2$ is in $\text{MAW}(W') \triangle \text{MAW}(W)$ for every $0 \leq \ell \leq k-2$ since b_1 is absent from W' . Also, β is in $\text{MAW}(W') \triangle \text{MAW}(W)$ and $b_{\pi-1} \alpha^\ell \beta$ is in $\text{MAW}(W') \triangle \text{MAW}(W)$ for every $0 \leq \ell \leq k-2$. Thus, at least $(\pi-3)k + k + 1 + (k-1) = (\pi-1)k$ MAWs are in $\text{MAW}(W') \triangle \text{MAW}(W)$. Additionally, the number $\pi-1$ of distinct characters which occur in W and are not equal to α is at least $\lfloor (\sigma-1)/2 \rfloor$, since $k \lfloor (\sigma-1)/2 \rfloor \leq k(\sigma-1)/2 = (k-k/2)(\sigma-1) \leq (k-1)(\sigma-1) \leq d$ where the second inequality follows from $k \geq 2$. Therefore, $|\text{MAW}(W') \triangle \text{MAW}(W)| \geq (\pi-1)k \geq \lfloor (\sigma-1)/2 \rfloor k \in \Omega(\sigma k) = \Omega(d)$. The number of pairs of two adjacent windows W and W' where $|\text{MAW}(W') \triangle \text{MAW}(W)| \in \Omega(d)$ is $\Theta((n-d)/k)$. Therefore, we obtain $\mathcal{S}(T', d) \in \Omega(d(n-d)/k) = \Omega(\sigma(n-d)) = \Omega(\sigma n)$ since $n-d \in \Omega(n)$. $\square$

Next, we consider the case where $\sigma \geq d+1$.

Lemma 9. *For a string T of length $n > d$ over an alphabet Σ of size σ , $\mathcal{S}(T, d) \in O(d(n-d))$, and this upper bound is tight when $\sigma \geq d+1$.*

Proof. By Theorem 2, it is clear that $\mathcal{S}(T, d) \in O(d(n-d))$. Next, we show that there is a string T' of length $n > d$ such that $\mathcal{S}(T', d) \in \Omega(d(n-d))$ for any integer d with $1 \leq d \leq \sigma-1$. Let $\Sigma = \{a_1, a_2, \dots, a_\sigma\}$. We consider a string $T' = (a_1 a_2 \dots a_{d+1})^e a_1 a_2 \dots a_k$ where $e = \lfloor n/(d+1) \rfloor$ and $k = n \bmod (d+1)$. For each window $W = T'[i..i+d-1]$ in T' , W consists of distinct d characters, and the character $T'[i+d]$ that is the right neighbor of W is different from any characters that occur in W . Without loss of generality, we assume that the current window is $W = a_1 a_2 \dots a_d$ and the next window is $W' = W[2..]a_{d+1} = a_2 \dots a_{d+1}$. Then, $|\text{MAW}(W') \triangle \text{MAW}(W)| = |\{a_1^2, a_{d+1}, a_2 a_1, \dots, a_d a_1, a_1 a_3, \dots, a_1 a_d\} \cup \{a_1, a_{d+1}^2, a_{d+1} a_2, \dots, a_{d+1} a_d, a_2 a_{d+1}, \dots, a_{d-1} a_{d+1}\}| = 4d-2 \in \Omega(d)$. Therefore, $\mathcal{S}(T', d) = d(n-d)$. $\square$

The main result of this section follows from the above lemmas:

Theorem 3. For a string T of length $n > d$ over an alphabet Σ of size σ , $S(T, d) \in O(\min\{d, \sigma\}n)$. This upper bound is tight when $n - d \in \Omega(n)$.

We remark that $n - d \in \Omega(n)$ covers most interesting cases for the window length d , since the value of d can range from $O(1)$ to cn for any $0 < c < 1$.

4 Tighter bounds for binary alphabets

In this section we consider the case where $\sigma' = 2$, i.e. when both the current sliding window $S = T[i..i + d - 1]$ and the next window $S\alpha = T[i..i + d]$ extended with a new character $\alpha = T[i + d]$ consist of two distinct characters. The goal of this section is to show that when $\sigma' = 2$, there exists a tighter upper bound for the number of changes of MAWs than the general case with $\sigma' \geq 3$.

In what follows, let us denote by $\Sigma_2 = \{0, 1\}$ the binary alphabet, and assume without loss of generality that we append the new character $\alpha = 0$ to the window S of length d and obtain the extended window $S\alpha = S0$.

As a warm up, we begin with the two following lemmas which show that at most 3 MAWs can change in the cases where $d = 1$ and $d = 2$ for any binary strings.

Lemma 10. For any string S over Σ_2 with $|S| = d = 1$, $|\text{MAW}(S) \triangle \text{MAW}(S0)| \leq 3$.

Proof. For each $S \in \{0, 1\}$ of length 1,

$$\begin{aligned} \text{MAW}(0) \triangle \text{MAW}(00) &= \{\underline{00}, 000\}, \\ \text{MAW}(1) \triangle \text{MAW}(10) &= \{\underline{0}, 00, 01\}, \end{aligned}$$

where the underlined strings are those in $\text{MAW}(S) \setminus \text{MAW}(S0)$ and the strings without underlines are those in $\text{MAW}(S0) \setminus \text{MAW}(S)$. Thus the lemma holds. $\square$

Lemma 11. For any string S over Σ_2 with $|S| = d = 2$, $|\text{MAW}(S) \triangle \text{MAW}(S0)| \leq 3$.

Proof. For each $S \in \{00, 01, 10, 11\}$ of length 2,

$$\begin{aligned} \text{MAW}(00) \triangle \text{MAW}(000) &= \{\underline{000}, 0000\}, \\ \text{MAW}(01) \triangle \text{MAW}(010) &= \{\underline{10}, 101\}, \\ \text{MAW}(10) \triangle \text{MAW}(100) &= \{\underline{00}, 000\}, \\ \text{MAW}(11) \triangle \text{MAW}(110) &= \{\underline{0}, 00, 01\}, \end{aligned}$$

where the underlined strings are those in $\text{MAW}(S) \setminus \text{MAW}(S0)$ and the strings without underlines are those in $\text{MAW}(S0) \setminus \text{MAW}(S)$. Thus the lemma holds. $\square$

We move onto the case where $d \geq 3$. Our first observation is that it is sufficient to consider the case that S is not unary. For any d , it is clear that $|\text{MAW}(0^d) \triangle \text{MAW}(0^{d+1})| = 2$. Now let us consider 1^d in the next lemma.

Lemma 12. For any $d \geq 3$ let $V = 1^d$. Then, there exists another string S of length d over Σ_2 such that $S[k] = 0$ for some $1 \leq k \leq d$ and $|\text{MAW}(V) \triangle \text{MAW}(V0)| \leq |\text{MAW}(S) \triangle \text{MAW}(S0)|$.

Proof. Since $V = 1^d$, $\text{MAW}(V) \setminus \text{MAW}(V0) = \{0\}$. Also, $\text{MAW}(V0) \setminus \text{MAW}(V) = \{00, 01\}$. Thus $|\text{MAW}(V) \triangle \text{MAW}(V0)| = 3$ for any $d \geq 1$.

Let $S = 01^{d-1}$ and $S0 = 01^{d-1}0$ with $d \geq 3$. Then, $\text{MAW}(S0) \setminus \text{MAW}(S) = \{01^k 0 \mid 1 \leq k \leq d-2\} \cup \{101\}$ and $\text{MAW}(S0) \setminus \text{MAW}(S) = \{10\}$. Thus we have $|\text{MAW}(S) \triangle \text{MAW}(S0)| \geq d \geq 3$. $\square$

According to Lemmas 10, 11 and 12, in what follows we focus on the case where $d \geq 3$ and the current window $S = T[i..i + d - 1]$ contains at least one 0. The latter condition implies that we focus on the case where the new character $\alpha = 0$ already occurs in the window S .

As in the case of non-binary alphabets, we analyze the numbers of added Type-1/Type-2/Type-3 MAWs in $\mathcal{M}_1/\mathcal{M}_2/\mathcal{M}_3$ for binary strings. Recall that in the current context, for any $S = T[i..i + d - 1]$, a MAW w in $\text{MAW}(S0) \setminus \text{MAW}(S)$ is said to be of:

- Type 1 if neither $w[2..]$ nor $w[..|w| - 1]$ occurs in S ;
- Type 2 if $w[2..]$ occurs in S but $w[..|w| - 1]$ does not occur in S ;
- Type 3 if $w[2..]$ does not occur in S but $w[..|w| - 1]$ occurs in S .

We first show the upper bound for the size of $\mathcal{M}_3$ in the case where $\sigma' = 2$.

Lemma 13. *For any binary string S over Σ_2 such that $|S| = d \geq 3$, $|\mathcal{M}_3| \leq d - 2$.*

Proof. Recall the proof for Lemma 4. There, we proved that each MAW w of Type 3 for any non-binary string $R\alpha = T[i..i + d] = T[i..j + 1]$ is mapped by an injection f to a distinct position of $T[i..j]$ in range $[i, j - 1]$, or alternatively to a distinct position of R in range $[1, d - 1]$. This showed $|\mathcal{M}_3| \leq d - 1$ for $\sigma' \geq 3$.

Here we show that the range of such an injection f is $[2, d - 1]$ for any binary string S with $\sigma' = 2$. Since the appended character is $\alpha = 0$, and since the candidate x for the MAW of Type 3 which should be mapped to the first position in S is of length 2, the candidate x has to be either 00 or 10.

- (1) If $x = 00$, then $S[1] = 0$. If 00 does not occur in S (see also the top picture of Figure 4), then 00 is already a MAW for S (i.e. $00 \in \text{MAW}(S)$). Thus $00 \notin \text{MAW}(S0) \setminus \text{MAW}(S)$ in this case. Otherwise (00 occurs in S), then clearly 00 is not a MAW for $S0$ (see also the middle picture of Figure 4).
- (2) If $x = 10$, then $S[1] = 1$. However, since the appended character is 0, 10 must occur somewhere in $S0$ (see also the bottom picture of Figure 4). Thus 10 is not a MAW for $S0$.

Hence, the first position of S cannot be assigned to any MAW of Type 3 for $S0$, leading to $|\mathcal{M}_3| \leq d - 2$ for any binary string S of length $d \geq 3$. $\square$

In other words, Lemma 13 shows that in the binary case with $\sigma' = 2$, the maximum number of added Type-3 MAWs is 1 less than in the case with $\sigma' \geq 3$.

Next, we consider the total number of added Type-1/Type-2 MAWs.

From Lemma 6, the next corollary holds.

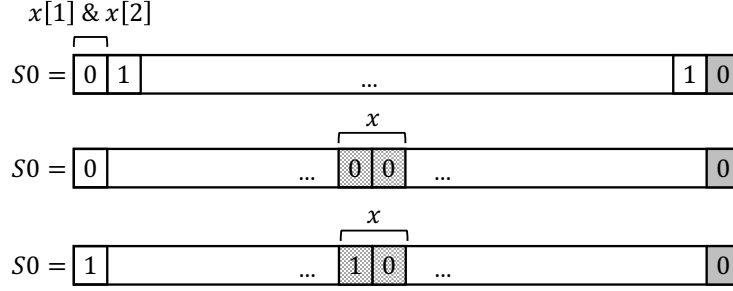


Figure 4: Characteristics of Type-3 MAWs in the binary case with $\sigma' = 2$, where the rightmost 0 in gray is the new appended character in each picture.

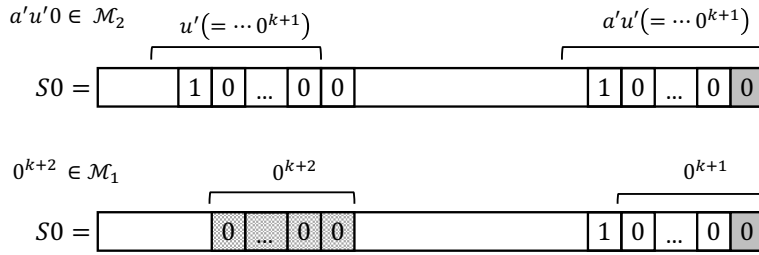


Figure 5: Collision between the new Type-2 MAW and Type-1 MAW in the binary case, where the rightmost 0 in gray is the new appended character in each picture.

Corollary 3. $|\mathcal{M}_1| + |\mathcal{M}_2| \leq 2$ on any binary string S when the character to be appended already occurs in S .

A direct consequence of Lemma 13 and Lemma 6 is an upper bound for the added MAWs $|\mathcal{M}_1| + |\mathcal{M}_2| + |\mathcal{M}_3| \leq d$ for any binary string $S0$ with $|S| = d \geq 3$. In what follows, we further reduce this upper bound to $|\mathcal{M}_1| + |\mathcal{M}_2| + |\mathcal{M}_3| \leq d - 1$. For this purpose, we introduce the next lemma:

Lemma 14. For any binary string S over Σ_2 such that $|S| = d \geq 3$, $|\mathcal{M}_2|$ is at most the number of occurrences of 0 in $S[1..d-1]$, and $|\mathcal{M}_3|$ is at most the number of occurrences of 1 in $S[3..d]$.

Proof. First we consider Type-2 MAWs for $S0$. By Lemma 3, there are at most two MAWs in $\mathcal{M}_2$. We assume that there are two MAWs in $\mathcal{M}_2$ and let $au0$ and $a'u'1$ be the two MAWs where $a, a' \in \Sigma_2$ and $u, u' \in \Sigma_2^*$. By the definition of Type-2 MAWs, $u0$ and $u'1$ occur in S and $u[u] = u'[u'] = 0$ since au and $a'u'$ occur in $S0$ as suffixes. For any two positions t_0 and t_1 such that $S[t_0..t_0 + |u0| - 1] = u0$, $S[t_1..t_1 + |u'1| - 1] = u'1$, $t_0 + |u| \neq t_1 + |u'|$. Consequently, there are at least two occurrences of 0 in S since $S[t_0 + |u| - 1] = 0$ and $S[t_1 + |u'| - 1] = 0$. Since $t_0 + |u0| - 1 \leq d$, $|\mathcal{M}_2|$ is at most the number of occurrences of 0 in $S[1..d-1]$.

Second we consider Type-3 MAWs for $S0$. let $au0$ be the Type-3 MAW where $a \in \Sigma_2$, $u \in \Sigma_2^*$ since $u0$ must be a suffix of $S0$. By the definition of Type-3 MAW, there has to be an occurrence of au in S . Note that this occurrence has to be immediately followed by a 1

since $au0$ does not occur in $S0$. Thus, for each $au0 \in \mathcal{M}_3$, we need an occurrence of $au1$ in S . Since $|au| \geq 1$, we clearly cannot use the first position of S as the ending position of $au1$. Also, it follows from Lemma 13 (and its proof) that the second position of S cannot be the ending position of au for any Type-3 MAW aub for $S0$. This implies that there is no Type-3 MAW that corresponds to the 1 in the second position of S . Thus, the total number of Type-3 MAWs for $S0$ is upper bounded by the number of occurrences of 1 in $S[3..d]$. $\square$

Intuitively, Lemma 14 implies that flipping substrings 1 in $S[3..d-1]$ does not increase the total number of Type-2 and Type-3 MAWs for $S0$.

Lemma 15. *For any binary string S over Σ_2 with $|S| = d \geq 3$, $|\mathcal{M}_1| + |\mathcal{M}_2| + |\mathcal{M}_3| \leq d - 1$.*

Proof. It immediately follows from Lemma 13 and Corollary 3 that $|\mathcal{M}_1| + |\mathcal{M}_2| + |\mathcal{M}_3| \leq d$, and assume on the contrary that there exists a binary string S' over Σ_2 such that $|\mathcal{M}_1| + |\mathcal{M}_2| + |\mathcal{M}_3| = d$. Then, it has to be $|\mathcal{M}_1| + |\mathcal{M}_2| = 2$ and $|\mathcal{M}_3| = d - 2$, again by Lemma 13 and Corollary 3. Therefore, $S'[3..d] = 1^{d-2}$ by Lemma 14, and $|\mathcal{M}_1| = 0$. Hence we must have $|\mathcal{M}_2| = 2$, which leads to $S' = 001^{d-2}$. Now the sets of all the added MAWs for $S'0 = 001^{d-2}0$ are

$$\begin{aligned}\mathcal{M}_1 &= \emptyset, \\ \mathcal{M}_2 &= \{100, 101\}, \\ \mathcal{M}_3 &= \{01^k0 \mid 1 \leq k \leq d-3\},\end{aligned}$$

which leads to $|\mathcal{M}_1| + |\mathcal{M}_2| + |\mathcal{M}_3| = d - 1$, a contradiction. Thus the lemma holds. $\square$

Our main theorem immediately follows from Lemma 1 and Lemma 15:

Theorem 4. *For any binary string S over Σ_2 with $|S| = d \geq 3$, $|\text{MAW}(S) \triangle \text{MAW}(S0)| \leq d$, and this upper bound is tight.*

The next corollary, which immediately follows Lemmas 10, 11, and Theorem 4, summarizes the results of this section.

Corollary 4. *For any binary string S over Σ_2 with $|S| = d$, $|\text{MAW}(S) \triangle \text{MAW}(S0)| \leq \max\{3, d\}$, and this upper bound is tight for any $d \geq 1$.*

5 Conclusions and future work

In this paper, we revisited the problem of computing the minimal absent words (MAWs) for the sliding window model, which was first considered by Crochemore et al. [10].

We investigated combinatorial properties of MAWs for a sliding window of fixed length d over a string of length n . Our contributions are *matching upper and lower bounds* for the number of changes in the set of MAWs for a sliding window when the window is shifted to the right by one character. For the general case where the window S and the extended window $S\alpha$ contain three or more distinct characters (i.e. $\sigma' \geq 3$), the number of changes in the set of MAWs for S and $S\alpha$ is at most $d + \sigma' + 1$ and this bound is tight. For the case of binary alphabets (i.e. $\sigma' = 2$), it is upper bounded by $\max\{3, d\}$ and this bound is also tight.

We also gave an asymptotically tight bound $O(\min\{d, \sigma\}n)$ for the number $\mathcal{S}(T, d)$ of total changes in the set of MAWs for every sliding window of length d over any string T of length n , where σ is the alphabet size for the whole input string T .

The following open questions are intriguing:

- We showed that a matching lower bound $\mathcal{S}(T, d) \in \Omega(\min\{d, \sigma\}n)$ when $n - d \in \Omega(n)$. Is there a similar lower bound when $n - d \in o(n)$?
- Crochemore et al. [10] gave an online algorithm that maintains the set of MAWs for a sliding window of length d in $O(\sigma n)$ time. Can one improve the running time to optimal $O(\min\{d, \sigma\}n)$?

Acknowledgments

This work was supported by JSPS KAKENHI Grant Numbers JP20J11983 (TM), JP18K18002 (YN), JP21K17705 (YN), JP17H01697 (SI), JP16H02783 (HB), JP20H04141 (HB), JP18H04098 (MT), and by JST PRESTO Grant Number JPMJPR1922 (SI).

The authors thank anonymous referees for their fruitful comments and suggestions for improving the presentation of the paper.

References

- [1] Y. Almirantis, P. Charalampopoulos, J. Gao, C. S. Iliopoulos, M. Mohamed, S. P. Pissis, and D. Polychronopoulos. On avoided words, absent words, and their application to biological sequence analysis. *Algorithms for Molecular Biology*, 12(1):5, 2017.
- [2] C. Barton, A. Heliou, L. Mouchard, and S. P. Pissis. Linear-time computation of minimal absent words using suffix array. *BMC Bioinformatics*, 15(1):388, 2014.
- [3] C. Barton, A. Heliou, L. Mouchard, and S. P. Pissis. Parallelising the computation of minimal absent words. In *PPAM 2015*, pages 243–253, 2016.
- [4] D. Belazzougui, F. Cunial, J. Kärkkäinen, and V. Mäkinen. Versatile succinct representations of the bidirectional Burrows-Wheeler transform. In *ESA 2013*, pages 133–144, 2013.
- [5] A. Blumer, J. Blumer, D. Haussler, A. Ehrenfeucht, M. T. Chen, and J. I. Seiferas. The smallest automaton recognizing the subwords of a text. *Theor. Comput. Sci.*, 40:31–55, 1985.
- [6] S. Chairungsee and M. Crochemore. Using minimal absent words to build phylogeny. *Theor. Comput. Sci.*, 450:109 – 116, 2012.
- [7] P. Charalampopoulos, M. Crochemore, G. Fici, R. Mercas, and S. P. Pissis. Alignment-free sequence comparison using absent words. *Inf. Comput.*, 262:57–68, 2018.
- [8] P. Charalampopoulos, M. Crochemore, and S. P. Pissis. On extended special factors of a word. In *SPIRE 2018*, pages 131–138. Springer, 2018.

- [9] T. Crawford, G. Badkobeh, and D. Lewis. Searching page-images of early music scanned with OMR: A scalable solution using minimal absent words. In *ISMIR 2018*, pages 233–239, 2018.
- [10] M. Crochemore, A. Héliou, G. Kucherov, L. Mouchard, S. P. Pissis, and Y. Ramusat. Absent words in a sliding window with applications. *Information and Computation*, 270:104461, 2020.
- [11] M. Crochemore, F. Mignosi, and A. Restivo. Automata and forbidden words. *Information Processing Letters*, 67(3):111–117, 1998.
- [12] M. Crochemore, F. Mignosi, A. Restivo, and S. Salemi. Data compression using antidictionaries. *Proc. IEEE*, 88(11):1756–1768, 2000.
- [13] M. Crochemore and G. Navarro. Improved antidictionary based compression. In *12th International Conference of the Chilean Computer Science Society, 2002. Proceedings.*, pages 7–13. IEEE, 2002.
- [14] G. Fici and P. Gawrychowski. Minimal absent words in rooted and unrooted trees. In *SPIRE 2019*, pages 152–161, 2019.
- [15] Y. Fujishige, Y. Tsujimaru, S. Inenaga, H. Bannai, and M. Takeda. Computing DAWGs and minimal absent words in linear time for integer alphabets. In *MFCS 2016*, volume 58, pages 38:1–38:14, 2016.
- [16] G. Koulouras and M. C. Frith. Significant non-existence of sequences in genomes and proteomes. *Nucleic acids research*, 49(6):3139–3155, 2021.
- [17] U. Manber and E. W. Myers. Suffix arrays: A new method for on-line string searches. *SIAM J. Comput.*, 22(5):935–948, 1993.
- [18] T. Mieno, Y. Kuhara, T. Akagi, Y. Fujishige, Y. Nakashima, S. Inenaga, H. Bannai, and M. Takeda. Minimal unique substrings and minimal absent words in a sliding window. In *SOFSEM 2020*, volume 12011 of *Lecture Notes in Computer Science*, pages 148–160. Springer, 2020.
- [19] D. Pratas and J. M. Silva. Persistent minimal sequences of sars-cov-2. *Bioinformatics*, 36(21):5129–5132, 2020.